

SPEC-2-OpenScholar-HLD

Background

OpenScholar is a scalable open-access academic publishing platform designed to democratize research access. The system must support fast MVP delivery while maintaining a clear path toward enterprise-grade scalability. This High-Level Design (HLD) defines the system architecture, major components, interactions, and technology choices for production deployment.

System Overview

OpenScholar is designed as a **Modular Monolith (Phase 1)** deployed on a VPS, with a clear migration path to **Microservices (Phase 2)**.

Key Goals

- Fast MVP delivery
 - Clean modular boundaries
 - Scalability readiness
 - High performance (<2s search)
-

Architecture Style

Phase 1: Modular Monolith

- Single deployable unit
- Internally separated modules
- Shared database

Phase 2: Microservices (Future)

- Independent services (Auth, Paper, Search, etc.)
 - Event-driven communication
-

High-Level Architecture

Components

1. **Frontend Layer**
2. Next.js (React-based)

3. Server-side rendering (SSR)
4. API routes (initial backend)
5. **Backend Layer (Application Logic)**

6. Next.js API routes (Phase 1)
7. Modular service layer
8. Future: NestJS services

9. **Database Layer**

10. PostgreSQL
11. Relational schema
12. Indexed search

13. **Storage Layer**

14. MinIO (S3-compatible)
15. Stores PDFs

16. **Reverse Proxy**

17. Nginx
18. Routing + SSL termination

Module Breakdown

Core Modules

- Auth Module
- User Module
- Paper Module
- Author Module
- Search Module
- Engagement Module
- Moderation Module
- Analytics Module
- Notification Module
- Storage Module

Module Responsibilities

Auth Module

- JWT authentication
- User sessions

Paper Module

- Submission
- Versioning
- Metadata handling

Search Module

- Full-text search
- Filtering

Engagement Module

- Comments
- Reactions

Analytics Module

- Event tracking
- Aggregation

Data Flow

Paper Upload Flow

Client → API → Storage → Database

Search Flow

Client → API → PostgreSQL (GIN index) → Response

Engagement Flow

Client → API → Database

API Gateway Design

- REST-based APIs

- Stateless communication
 - JWT authentication
-

Database Design Strategy

- Normalized schema
 - Version-controlled papers
 - Junction tables for relationships
 - Soft delete support
-

Storage Design

Phase 1

- MinIO on VPS

Phase 2

- AWS S3
-

Security Architecture

- HTTPS via Nginx
 - JWT authentication
 - Password hashing (bcrypt)
 - RBAC enforcement
-

Performance Strategy

- PostgreSQL GIN index for search
 - Aggregated metrics table
 - Efficient query design
-

Scalability Strategy

Vertical Scaling (Phase 1)

- Increase VPS resources

Horizontal Scaling (Phase 2)

- Microservices
 - Load balancer
 - Distributed storage
-

Deployment Architecture

VPS Setup

- Ubuntu server
- Docker containers
- Nginx reverse proxy

Components

- Next.js app
 - PostgreSQL
 - MinIO
-

Monitoring & Logging

- Application logs
 - Error tracking
 - Performance monitoring
-

Failure Handling

- Retry mechanisms
 - Graceful degradation
 - Backup and recovery
-

Future Enhancements

- Elasticsearch integration
 - Redis caching
 - Queue system (Kafka/RabbitMQ)
 - Microservices migration
-

Risks & Mitigation

Risk	Mitigation
Large file upload issues	Streaming + storage abstraction
Search slowdown	Indexing + ElasticSearch
Scaling bottlenecks	Modular design + migration path

Conclusion

This HLD provides a production-ready architectural foundation for OpenScholar, enabling rapid MVP delivery while ensuring long-term scalability, maintainability, and performance.